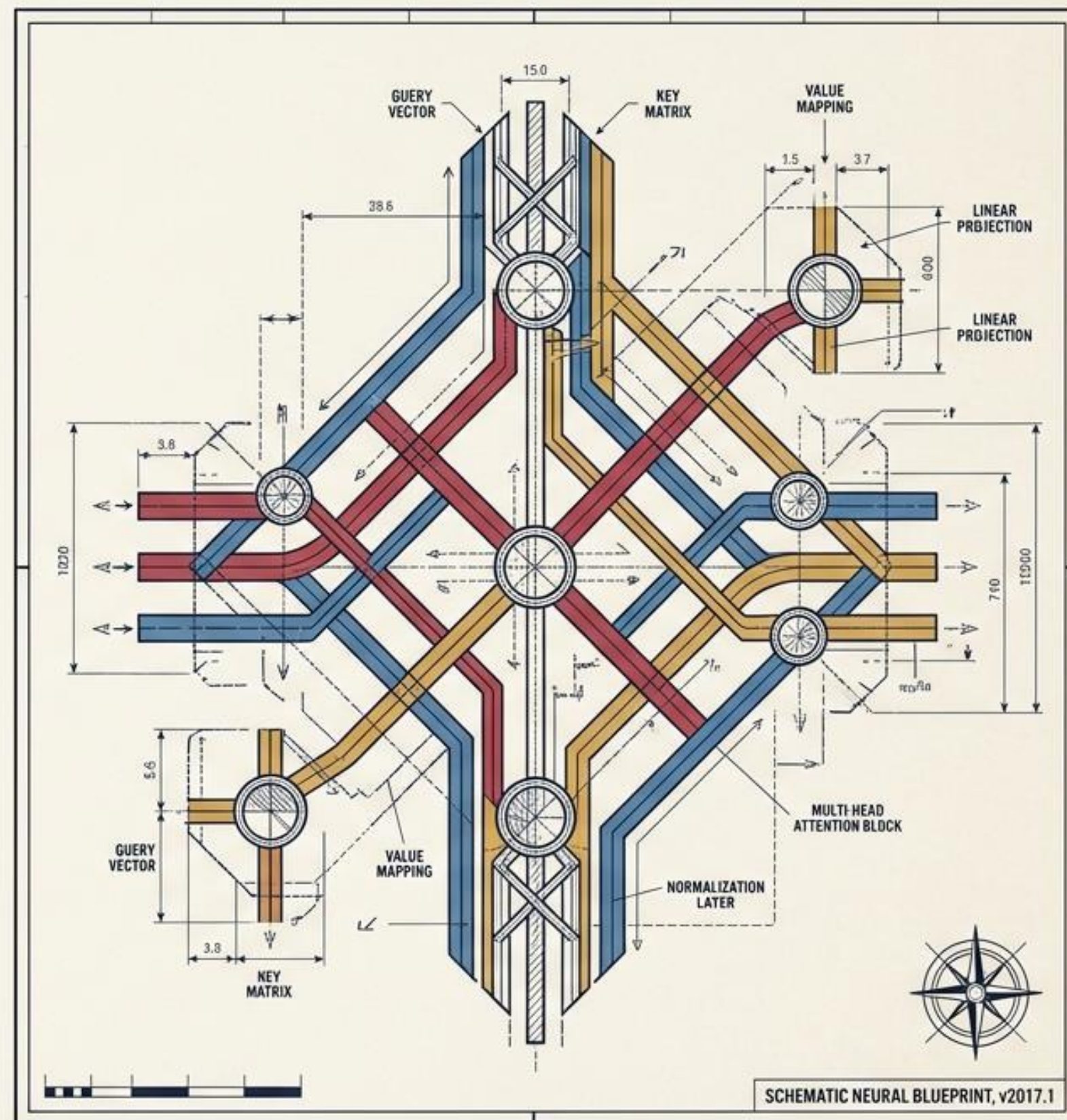


ATTENTION IS ALL YOU NEED

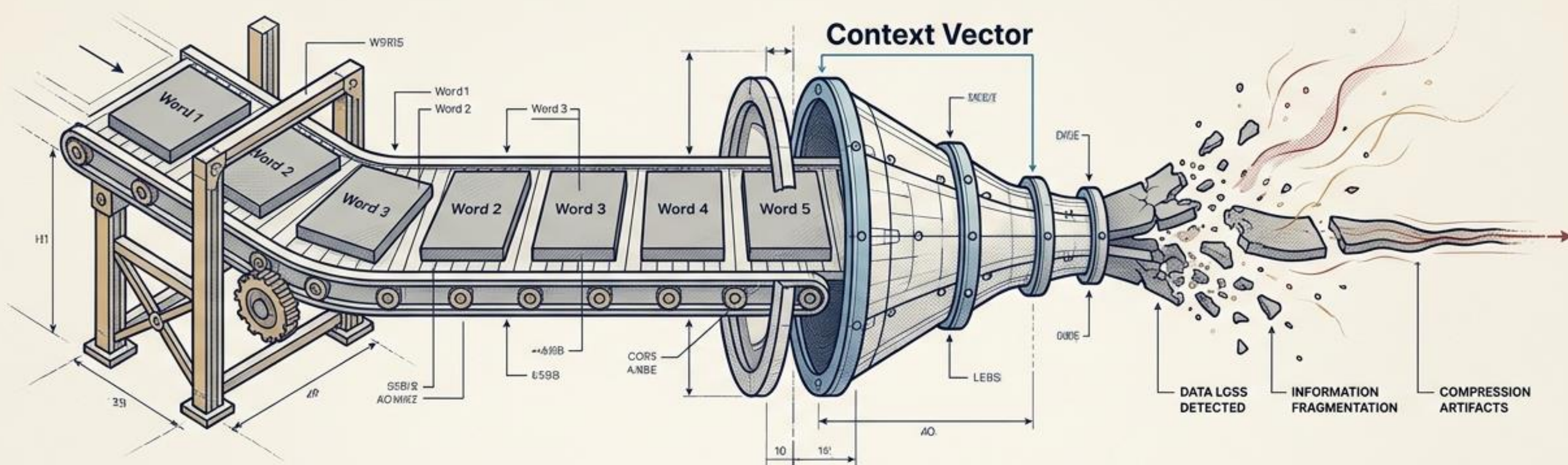
현대 AI의 패러다임을 바꾼...

2017년 구글이 발표한 이 논문은 기존 인공지능의 한계를 깨고, 현재의 GPT, BERT 등 거대 언어 모델(LLM) 시대를 연 핵심 아키텍처 '트랜스포머(Transformer)'를 세상에 처음 공개했습니다.

이 문서는 복잡한 수식 이면에 숨겨진 트랜스포머의 기계적 작동 원리를 시각적으로 해부하는 정밀 해설서입니다.

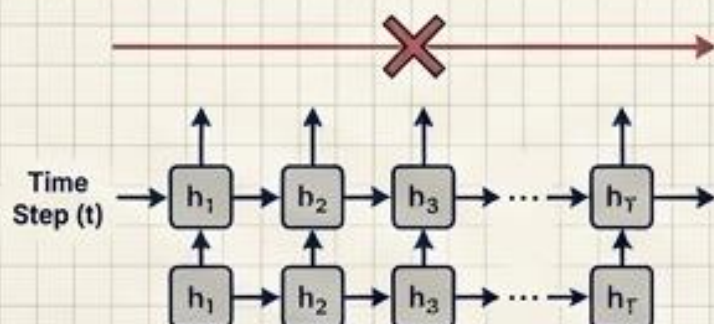


과거 모델의 치명적 한계: 정보 압축의 병목 현상



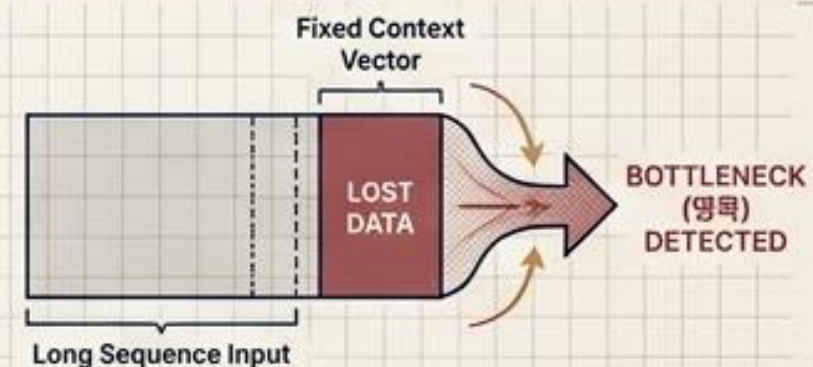
순차 처리의 한계 (Sequential Processing)

기존 RNN과 LSTM 기반 모델은 단어를 하나씩 순서대로 읽어야만 했습니다. 이는 데이터 처리를 느리게 만들고, 최신 GPU의 병렬 처리 능력을 활용할 수 없게 만들었습니다.

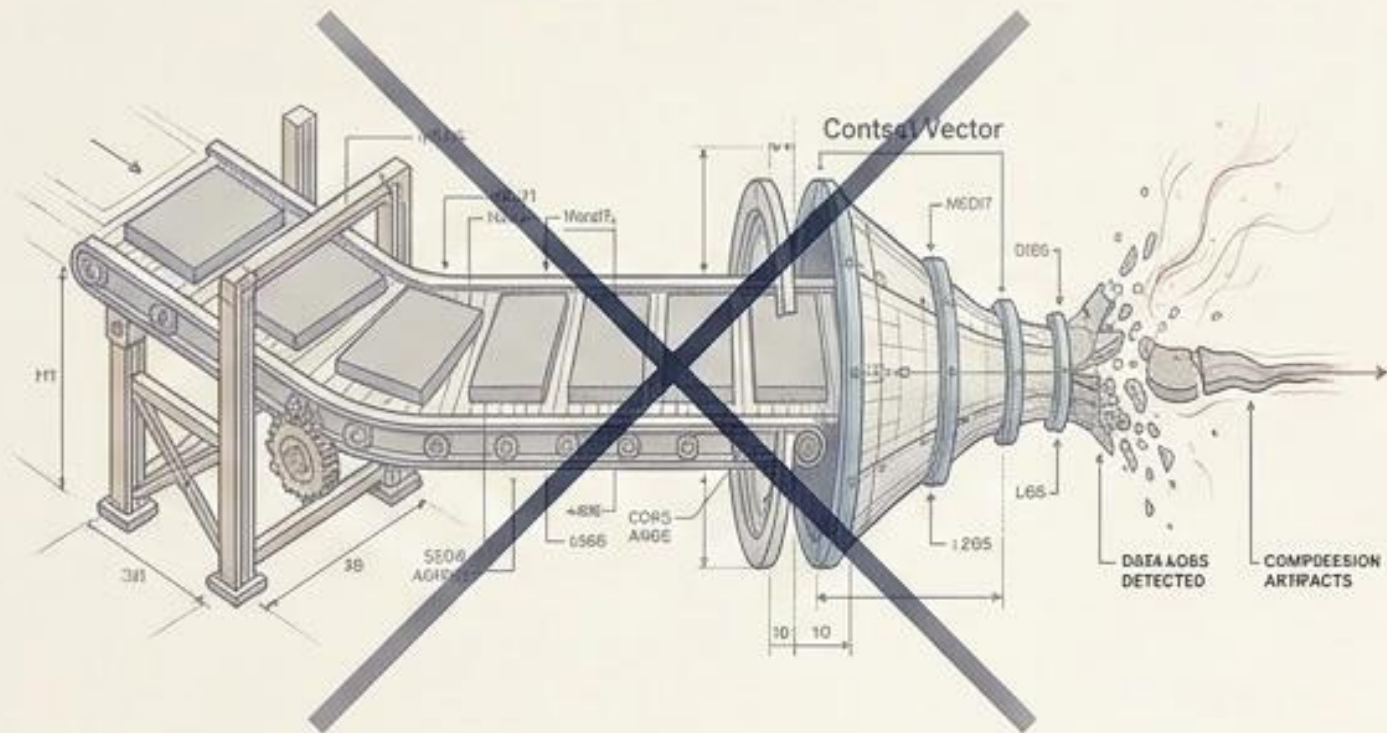


장기 의존성 문제 (Long-Term Dependency)

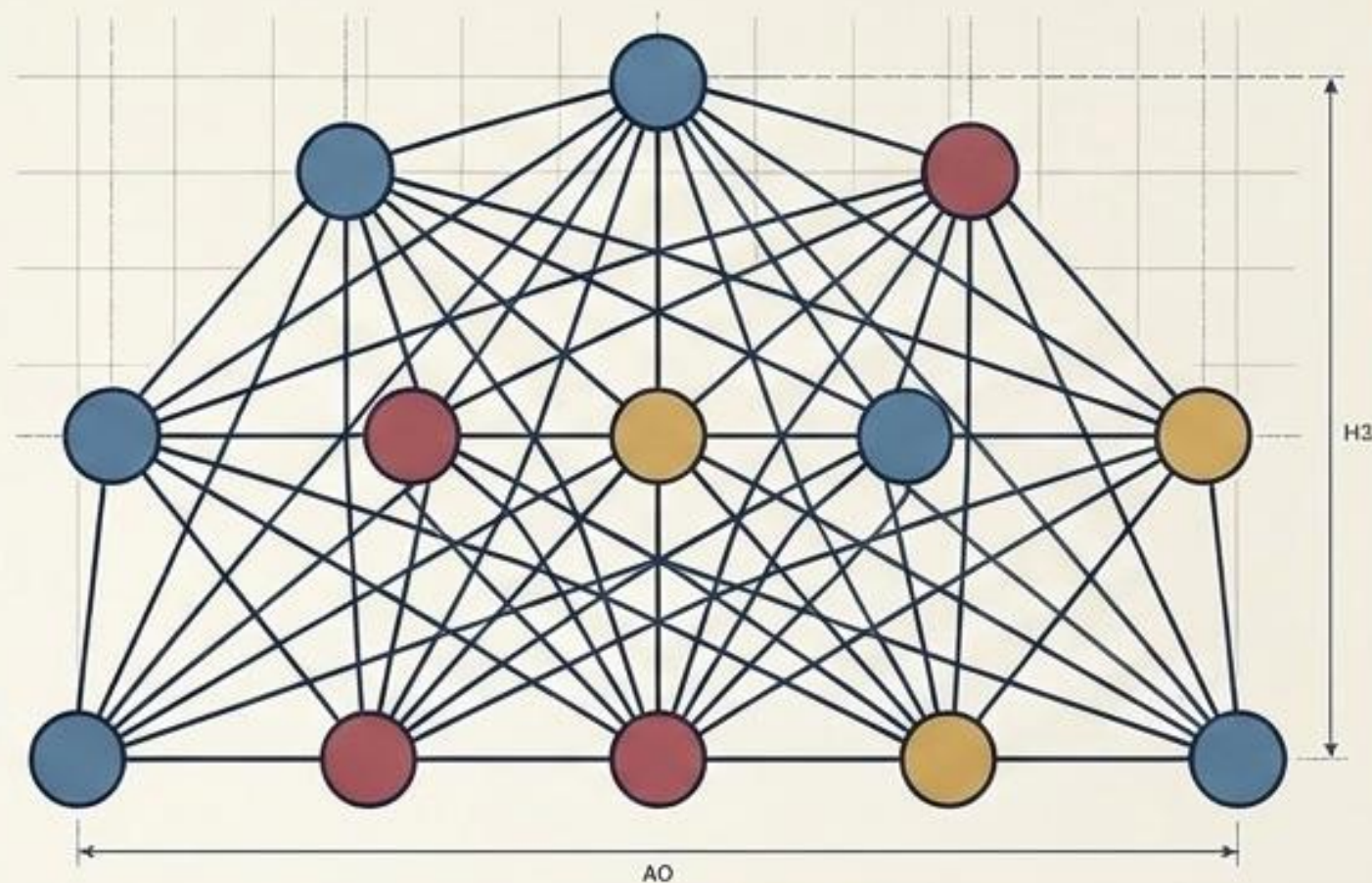
모든 입력 문장의 정보를 고정된 크기의 단일 Context Vector에 억지로 압축해야 했습니다. 문장이 길어질수록 초반에 입력된 정보가 유실되는 병목(Bottleneck) 현상이 발생했습니다.



패러다임의 전환: 순서를 버리고 '연결'을 택하다



BEFORE



AFTER - The Transformer

NO RNN, NO CNN

트랜스포머는 순환 신경망(RNN) 구조를 완전히 과감하게 버렸습니다.

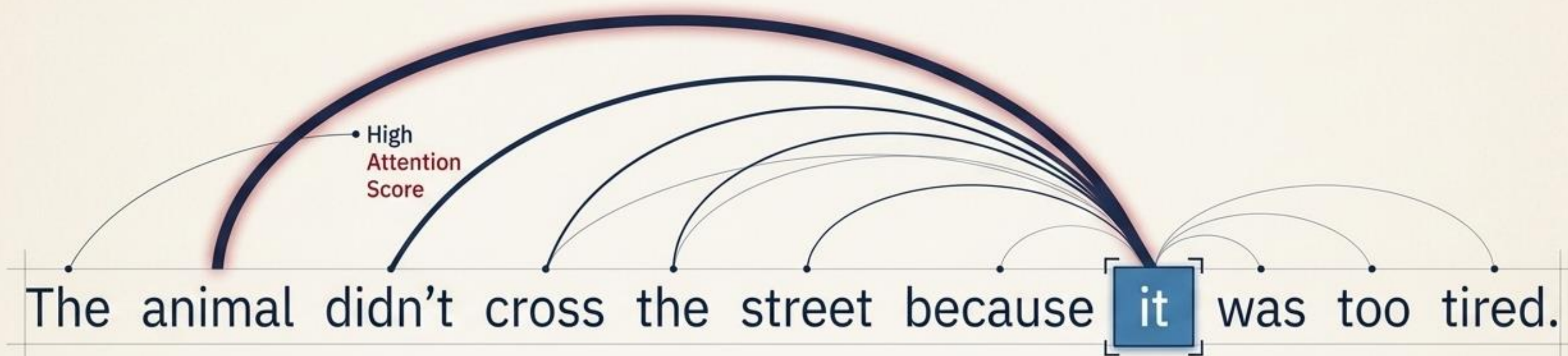
병렬화 극대화

한 번의 행렬 연산으로 시퀀스 전체를 동시에 처리하여 연산 속도를 기하급수적으로 끌어올렸습니다.

Attention Only

오직 '어텐션(Attention)' 메커니즘 하나에만 전적으로 의존하여 전역적인 관계를 학습합니다.

문맥을 이해하는 힘: Self-Attention



질문

문장에서 'it'이 가리키는 것은 'street(길)'일까요, 'animal(동물)'일까요? 인간에게는 쉽지만, 기계에게는 어려운 문제입니다.

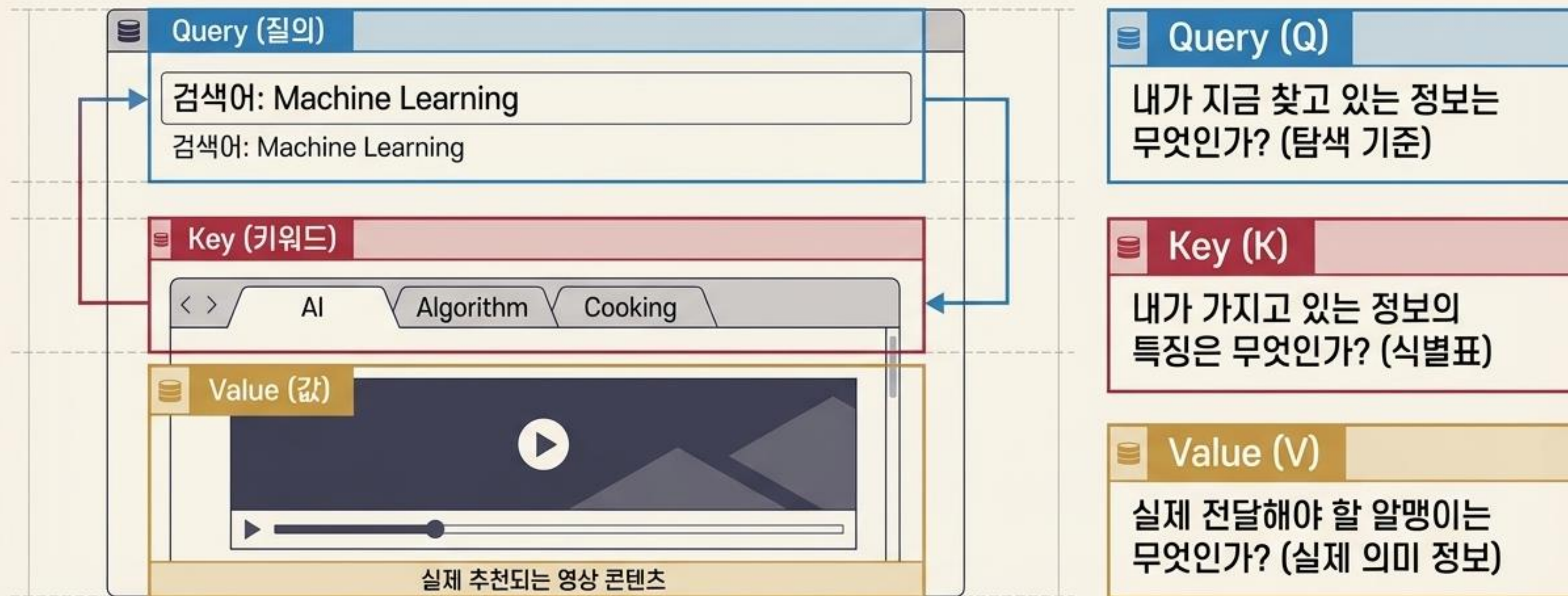
해결책

Self-Attention은 하나의 문장 내에서 단어와 단어 사이의 관계를 직접 계산합니다.

동작 원리

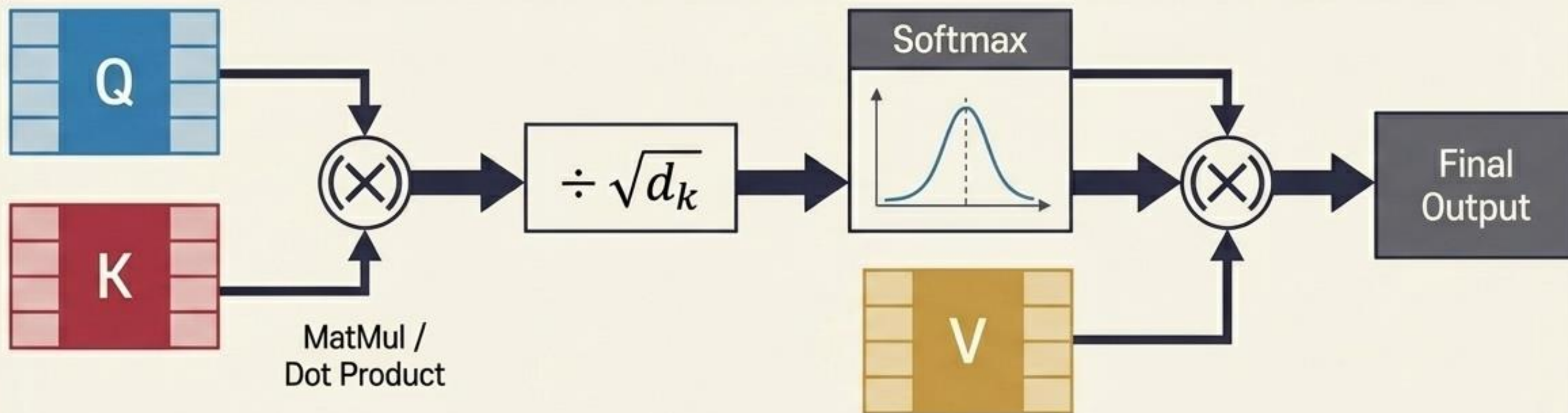
모델이 'it'을 처리할 때, 문장 내의 모든 다른 단어들을 스캔하여 가장 연관성 높은 단어(animal)에 높은 가중치를 부여하고, 그 의미를 'it'의 벡터에 융합합니다.

어텐션의 본질: '검색 엔진' 메커니즘 (Q, K, V)



Query(질의) × Key(키) = 연관성(Score) → 연관성만큼 Value(값)를 가져와 합산.

Scaled Dot-Product Attention: 수학적 해부



1. 행렬 내적

Q 행렬과 K 행렬을 곱하여 단어 간의 유사도 점수를 계산합니다.

2. 스케일링

Key 벡터의 차원 수로 점수를 나눕니다. 값이 너무 커져 기울기가 소실되는 현상을 방지합니다.

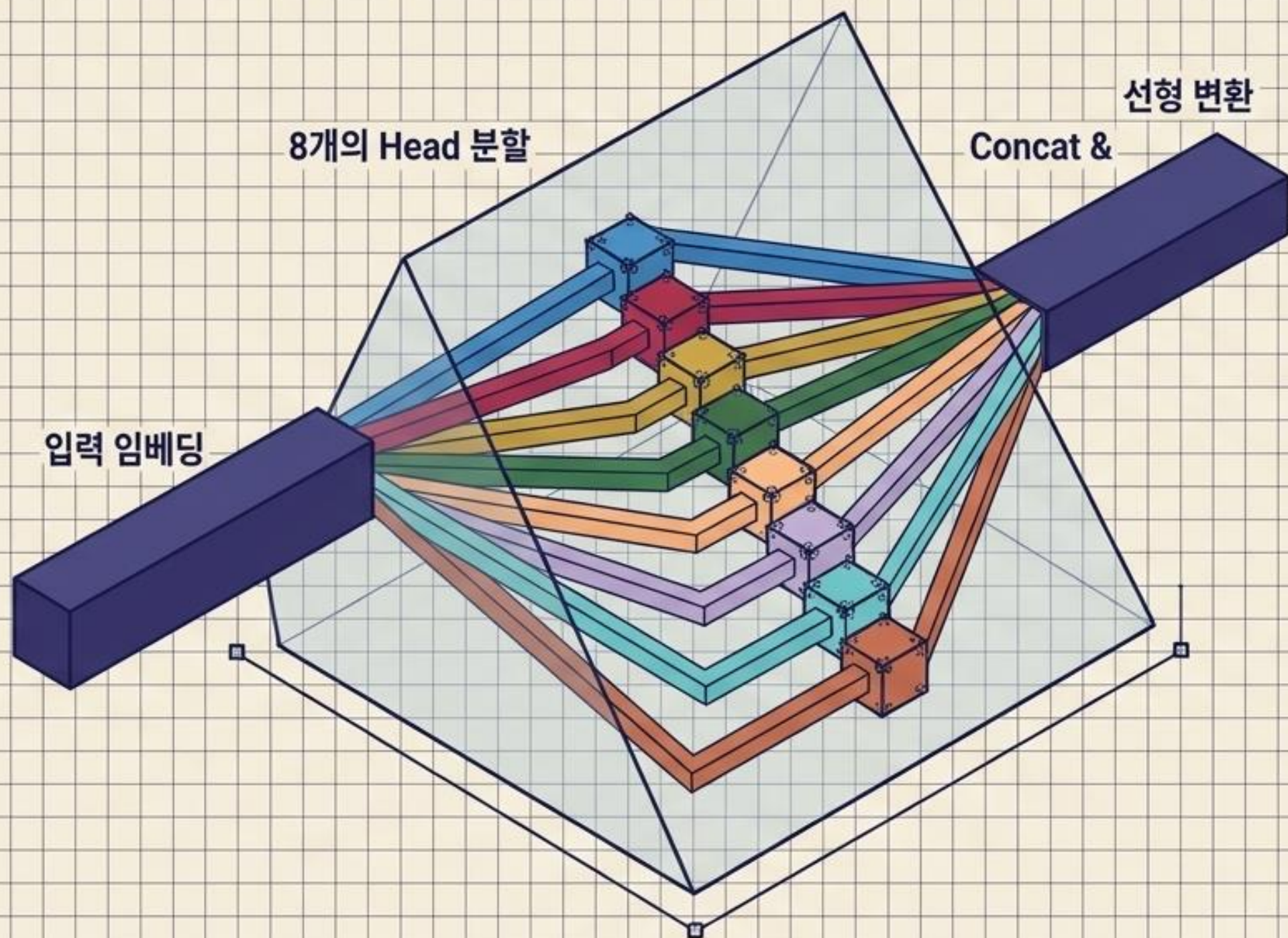
3. 소프트맥스

점수들을 양수로 정규화하여 총합이 1(100%)이 되는 가중치 비율로 변환합니다.

4. 가중합

도출된 가중치를 실제 정보인 V 벡터에 곱하여 최종 결괏값을 산출합니다.

다각도로 문장 바라보기: Multi-Head Attention



❏ 프리즘 메타포

빛이 프리즘을 통과하며 여러 스펙트럼으로 나뉘듯, 임베딩 차원을 8개의 작은 공간(Sub-space)으로 분할합니다.

❏ 왜 헤드를 여러 개 두는가?

단일 어텐션은 하나에만 집중합니다. 멀티 헤드를 사용하면 '1번 헤드는 문법 구조, 2번 헤드는 감정, 3번 헤드는 주어-동사 관계'를 동시에 병렬로 학습할 수 있습니다.

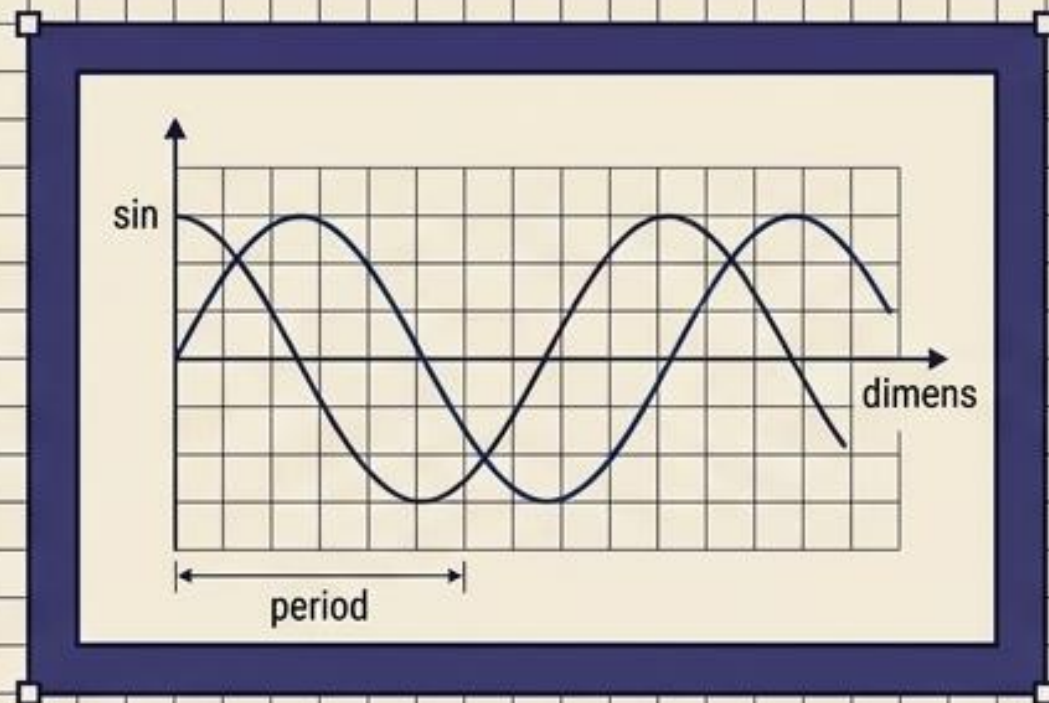
❏ 비용 효율성

차원을 나누어 연산하므로, 8번을 수행해도 전체 연산량은 단일 어텐션과 유사하게 유지됩니다.

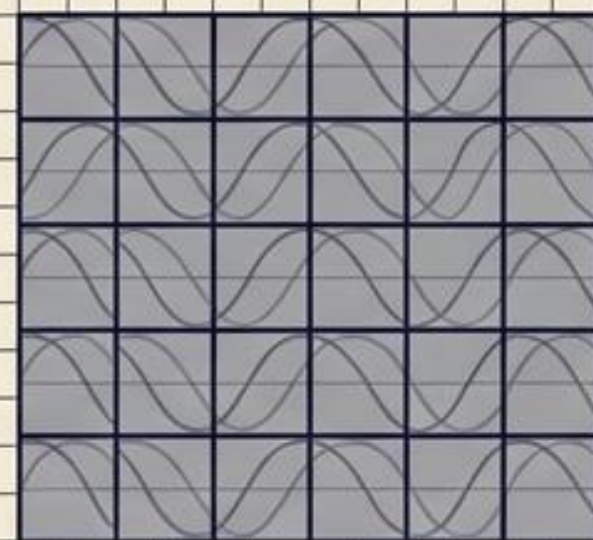
공간에 시간을 부여하다: Positional Encoding



+



=



[Word Embedding: 공간/의미 정보]

[Positional Encoding: 시간/순서 정보]

[Final Input: 순서가 각인된 임베딩]



문제:

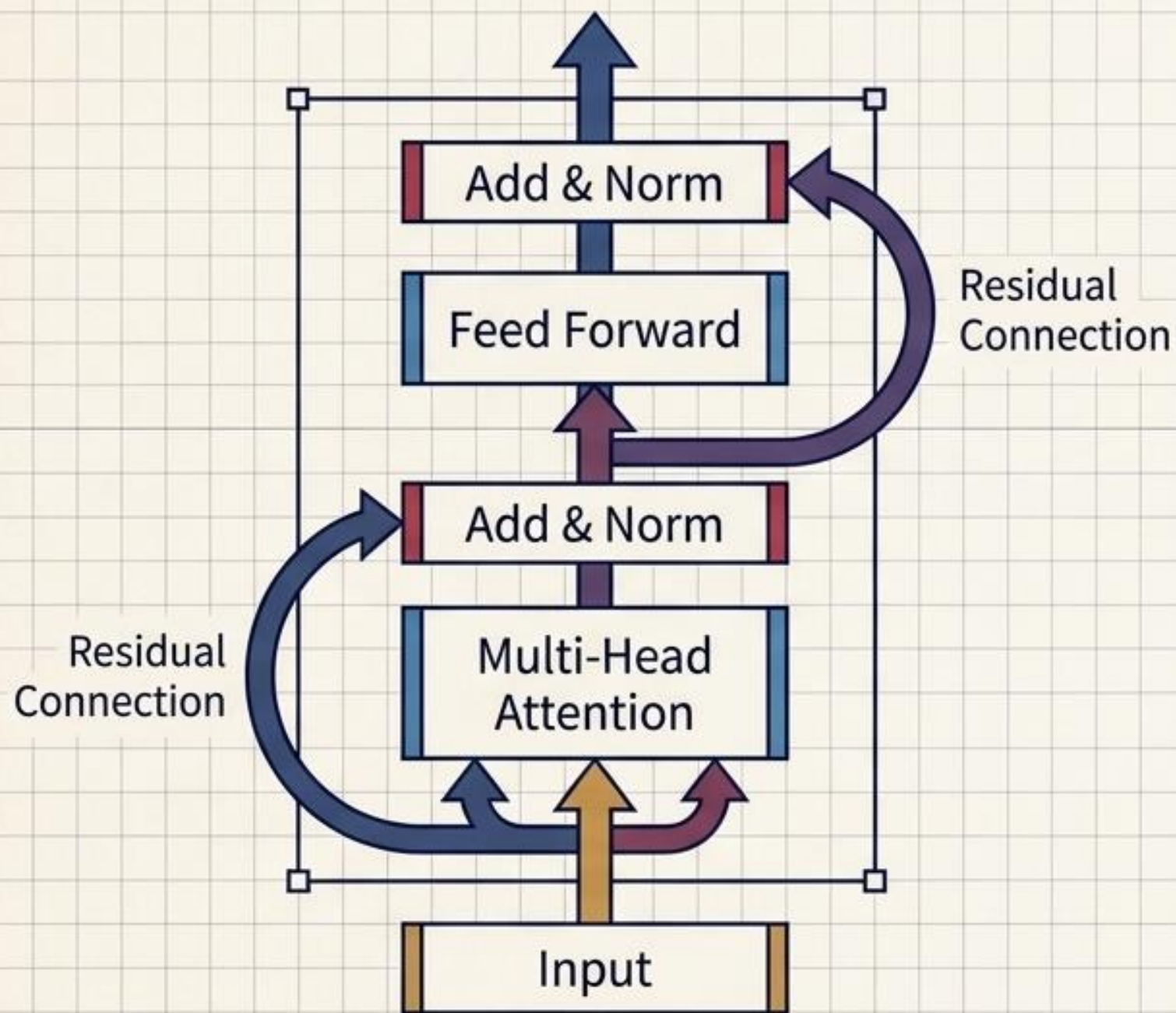
트랜스포머는 단어들을 동시에 병렬로 밀어 넣습니다.
기계는 단어의 '순서(Time/Order)'를 알 수 없게 됩니다.



해결책:

RNN을 쓰지 않는 대신, 사인(Sine)과 코사인(Cosine) 주기 함수를 이용해 각 단어의 고유한 '타임스탬프' 좌표를 수학적으로 생성합니다. 이 위치 벡터를 단어 벡터와 더하여(+) 순서 정보를 모델에 주입합니다.

이해의 층을 쌓다: The Encoder Stack



6개의 레이어 ($N \times 6$)

트랜스포머의 인코더는 이 구조를 6번 반복하여 입력 문장에 대한 깊고 정교한 문맥 지도를 구축합니다.

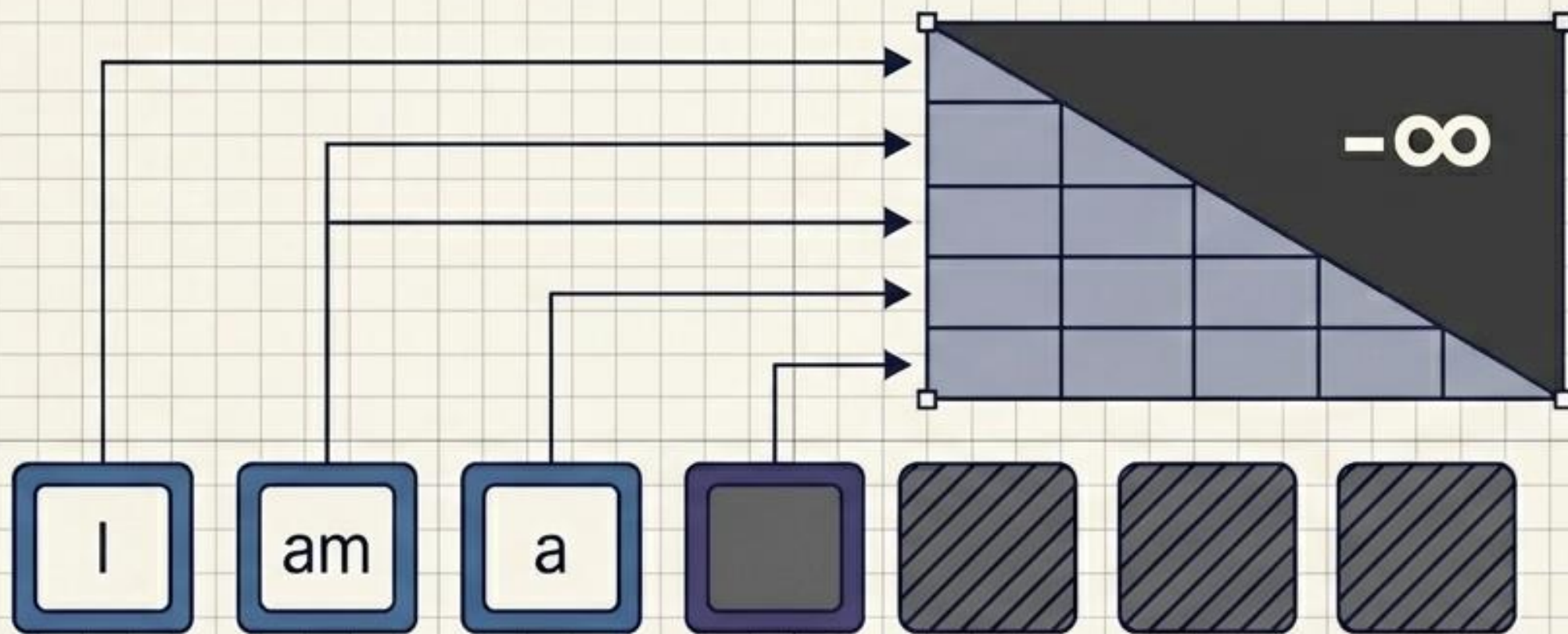
잔차 연결 (Residual Connection)

정보가 레이어를 통과하며 손실되는 것을 막기 위해, 원본 입력값을 레이어 결깃값에 직접 더해줍니다. 안정적인 학습을 보장합니다.

피드 포워드 (Feed Forward)

어텐션을 통해 모인 문맥 정보를 각 위치별로 독립적이고 동일하게 비선형 변환하여 표현력을 증폭시킵니다.

미래를 가린 채 새로운 언어를 창조하다: The Decoder



순차적 생성 (Auto-Regressive)

인코더가 전체를 한 번에 이해한다면, 디코더는 이전까지 생성된 단어들을 바탕으로 다음 단어를 '하나씩' 예측합니다.



Masked Self-Attention

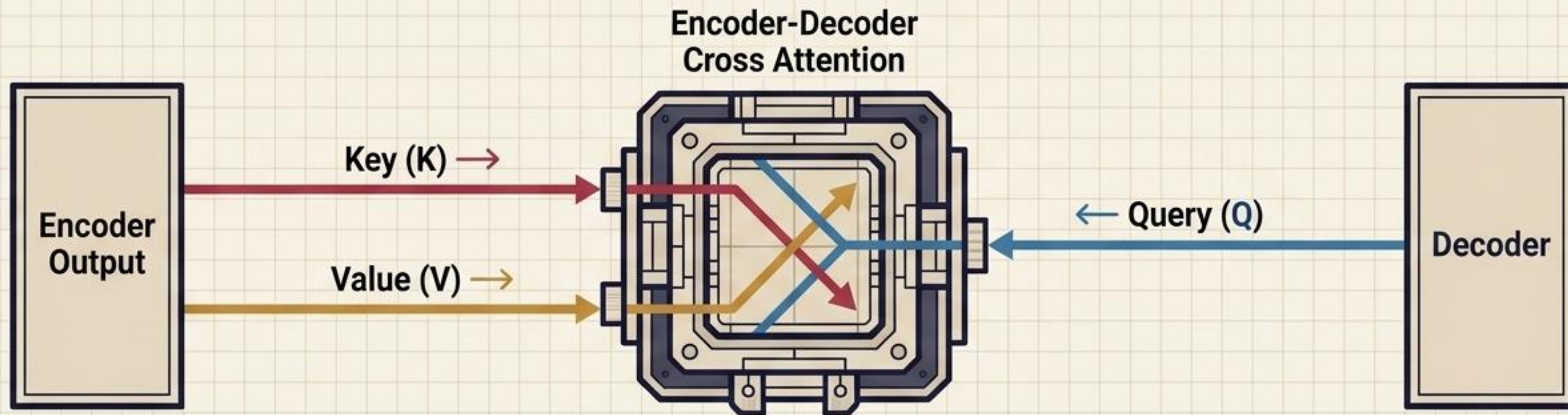
학습 중 디코더가 정답(미래의 단어)을 미리 '컨닝(Cheating)'하는 것을 방지해야 합니다.



동작 방식

모델이 미래 시점의 단어를 참조하지 못하도록 Attention Score 행렬의 해당 위치에 $-\infty$ 마스크를 씌워, Softmax 결과값이 0이 되게 만듭니다.

두 세계를 연결하는 다리: Encoder-Decoder Attention



Decoder's Role

Query (질의)는 디코더에서 디코더는 현재 생성 단계의 기준을 묻습니다.
'방금 이전 단어들을 번역했어. 다음 단어를 만들려면 원본에서 뭘 봐야 해?'

Encoder's Role

Key & Value (키 & 값)는 인코더에서 인코더는 원본 문장의 맥락을 제공합니다.
'원본 문장의 모든 정보는 여기 있어. 연관성을 계산해서 필요한 만큼 가져가.'

압도적인 게임 체인저: 왜 트랜스포머인가?

	Layer Type	순차 연산 횟수 (Sequential Ops)	최대 경로 길이 (Max Path Length)
1	Recurrent (RNN)	$O(n)$	$O(n)$
2	Convolutional (CNN)	$O(1)$	$O(\log_k(n))$
3	Self-Attention (Transformer)	$O(1)$	$O(1)$

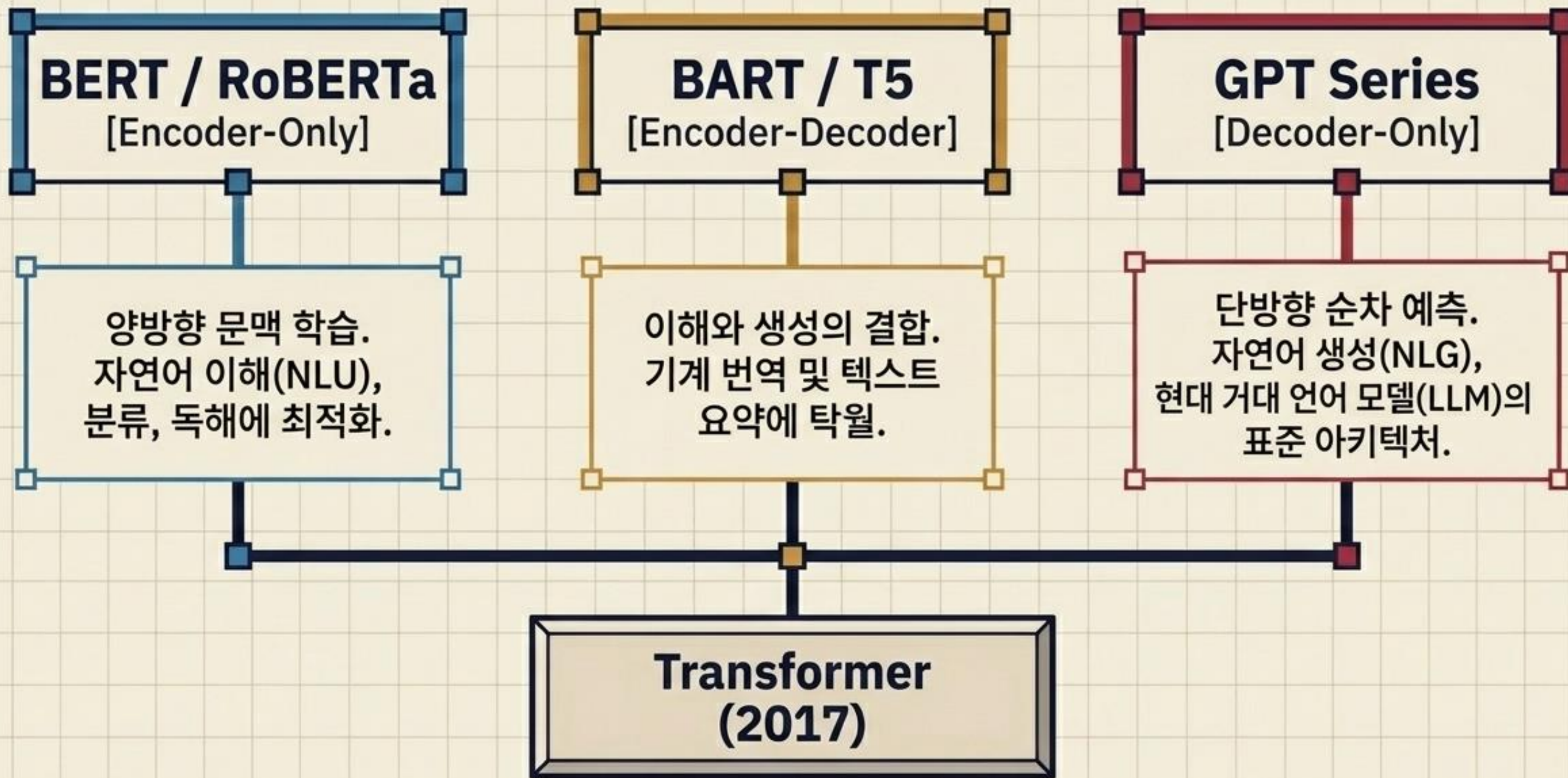
완벽한 병렬화와 최단 경로

RNN은 단어 수만큼 지연이 발생하지만, 트랜스포머는 $O(1)$ 의 복잡도로 완벽한 병렬화와 모든 단어 간의 직접 연결을 이뤄냈습니다.

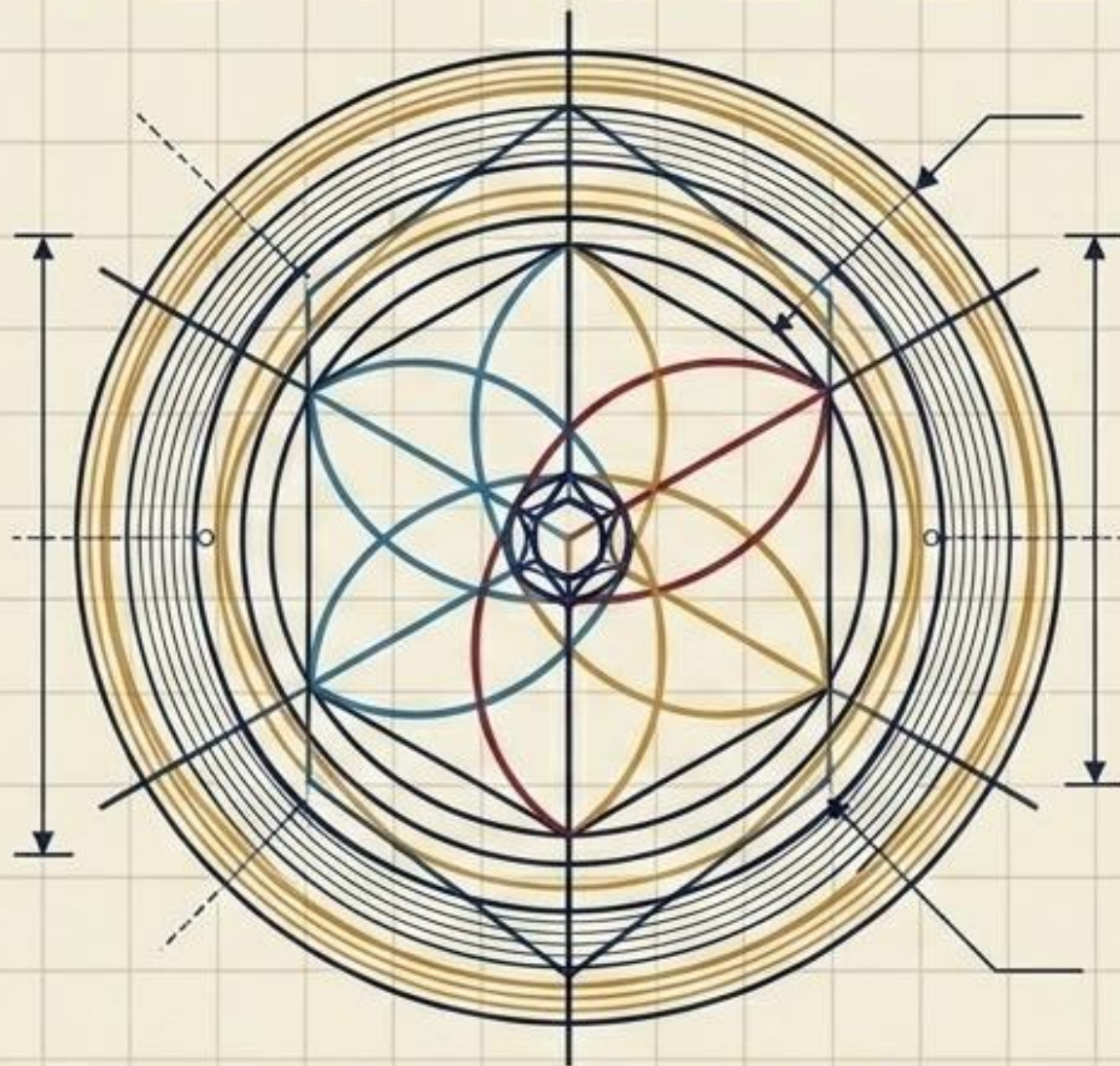
압도적인 성능 결과 (WMT 2014)

- 영어-독일어: 28.4 BLEU (기존 SOTA 경신)
- 학습 비용: 단 8개의 P100 GPU로 12시간 만에 학습 완료 (기존 대비 1/4 비용)

현대 AI 생태계의 근간: Transformer Heritage



연결의 힘이 지능을 만듭니다.



트랜스포머는 데이터를 처리하는 방식을 '순서(Sequence)'에서 '관계(Attention)'로 바꾸어 놓았습니다.
단어와 단어가 서로에게 얼마나 주의를 기울이는지 계산하는 단순하지만 강력한 수학적 은유는,
기계가 인간의 언어를 진정으로 조망할 수 있게 한 게임 체인저였습니다.

Attention Is All You Need.